
CASE STUDY REPORT

TITLE

Git-Based Collaborative Software Development Workflow:
A Case Study of the QNext Hospital Queue Management System
(Ashva Technologies Pvt. Ltd.)

Maitri Adroja [92400103446]
Krupa Makwana [92400103408]
Manthan Makwana [92400103407]

DEPARTMENT
COMPUTER ENGINEERING

ACADEMIC YEAR
2026-2027

SEMESTER
5TH

SUBJECT
OPEN SOURCE TECHNOLOGIES (01CE0526)

CLASS
5EC3

BATCH
A

CASE STUDY REPORT

RUBRICS FOR CASE STUDY

Sr. No.	Assessment Criteria	Excellent	Good	Satisfactory	Poor	Marks
1	Problem Analysis, Cloud Concepts & SDG Mapping	4 Marks: Clear problem identification, appropriate cloud concepts, and accurate SDG mapping.	3 Marks: Good analysis with minor gaps in concepts or SDG mapping.	1–2 Marks: Basic problem understanding with partial cloud concepts or SDG mapping.	0 Marks: Inadequate or incorrect analysis and no SDG mapping.	4
2	Security Analysis & Technical Depth	4 Marks: Comprehensive security analysis, well-defined architecture, appropriate technologies, and justified solutions.	3 Marks: Good technical explanation with minor omissions.	1–2 Marks: Limited security analysis and technical details.	0 Marks: No meaningful security analysis or technical explanation.	4
3	Report Quality & References	4 Marks: Well-structured report in IEEE format with comprehensive technical content, proper citations, and relevant references.	3 Marks: Good report structure with minor formatting or referencing issues.	1–2 Marks: Basic report with limited technical details or incomplete references.	0 Marks: Incomplete report with poor documentation and missing references.	4
	Total					12

Sr. No.	Case Study Title / Problem Statement	COs	Date	R1(4)	R2(4)	R3(4)	Marks (12)	Signature
1	Git-based Collaborative Software Development Workflow							

Notes / Instructions

- The case study must be completed in a **group of maximum 3 students**.
- Students must follow the **given submission deadlines**.
- The case study report must follow the **provided IEEE Conference Paper Template format**.
- The final report must be uploaded in **DOCX format on Google Classroom**, and a **hard copy must be submitted** as discussed.
- The report should include proper **technical analysis, cloud concepts, security aspects, SDG mapping, figures/tables, and IEEE-style references**.
- Use only **authentic research papers and reliable sources** for references.
- All group members must contribute equally and be prepared for **presentation and evaluation**.
- Remove all template instructions/content before final submission.
- Plagiarism or copied content will not be accepted.

Ms. Priyanka Mangi
Subject Faculty

CASE STUDY REPORT

TITLE

Course Name and Course Code: OPEN SOURCE TECHNOLOGIES (01CE0526)

Co's Mapped - Version Control Tools & Collaboration

SDG's Mapped - SDG 9 – Industry, Innovation and Infrastructure

SDG 17 – Partnerships for the Goals

**FORMAT OF CASE STUDY IS ALREADY SHARED WITH YOU IN IEEE CONFERENCE
TEMPLATE STYLE USE THAT ONLY**

Abstract
Keywords

I. Introduction

- Background of the selected case study
- Importance of Git and GitHub Software for Development
- Objectives of the case study
- Scope of the study

II. Company Background

- Company Overview
- QNext Project
- Business requirements for Collaborative Development
- Need for Utilizing a Git Workflow

III. Problem Analysis

- Current Development Process (Pre-Git)
- Problems Encountered
- Engineering requirements
- Functional and non-functional requirements
- Constraints and assumptions

IV. Git and GitHub Solution Analysis

- Chosen Tools: Git & GitHub
- Why Git and GitHub were Chosen
- Licensing (GPLv2)
- Comparison with alternative approaches
- Advantages of Using Git and Github

V. System Architecture and Branching Strategy

- General Git Cooperation Procedure
- Branching Strategy for QNext
- Creation of Local and Remote Repositories

VI. Team Structure and Repository Structure

- Team Structure
- Repository File Organization
- Branch Protection and commit Message Writing

CASE STUDY REPORT

VII. Feature Development, Pull Request and Code Review Process

- Feature Development: Token Booking and Queue Dashboard
- Creating and Pushing Feature Branches
- Pull Request Creation
- Code Review Process

VIII. Merge Conflict Scenario and Resolution

- Cause of the Merge Conflict
- Sequence of Events
- File Involved and Conflicting Changes
- Problem Solution

IX. Testing and Version Control Practices

- Testing the Resolve Code
- Tracking Versions and Performing Revert
- Tags and Release

X. CO2 and SDG Mapping

- CO2- Version Control Tools and Collaboration
- SDG 9- Industry, Innovation and Infrastructure
- SDG 17- Partnerships for the Goals

XI. Results, Advantages and Challenges

- Identified Results
- Advantages
- Problems

XII. Key Learning and Conclusion

- Summary of the case study
- Technical Insights
- Learning outcomes
- Industrial relevance

References

- IEEE Reference Format
- Minimum 5–8 authentic references
- Research papers
- Official Documentation
- GitHub Documentation
- Linux Foundation Documentation
- Books

CASE STUDY REPORT

A Case Study of the QNext Hospital Queue Management System (Ashva Technologies Pvt. Ltd.)

Maitri Adroja
Department Of Computer
Engineering

Marwadi University
Rajkot, India

maitri.adroja130626@marwadi
university.ac.in

Krupa Makwana
Department Of Computer
Engineering

Marwadi University
Rajkot, India

krupa.makwana130100@marwadi
university.ac.in

Manthan Makwana
Department of Computer
Engineering

Marwadi University
Rajkot, India

manthan.makwana130093@marwa
diuniversity.ac.in

Abstract— Most modern software applications are no longer developed by individual developers alone. Small teams of developers collaboratively develop software by coding, reviewing, and integrating codes together even as they modify the same portions of the same file at the same time. This paper highlights how a fictional composite software firm, Ashva Technologies Pvt. Ltd., employed a Git based collaborative workflow during development of QNext, a token and queue management system for the outpatient department of a government hospital. It follows the transition of the development team from an informal process of development through sharing of code through zip files, which was responsible for loss of one day of coding work by one developer, to a formalized workflow of using feature branches, pull requests and code review on GitHub. The workflow is illustrated in detail by analysis of a realistic merge conflict situation where two developers separately modify the same queue ordering code. The paper illustrates how the workflow maps to Course Outcome CO2 (Version Control Tools and Collaboration) and Sustainable Development Goals 9 and 17. and the report closes with the practical benefits and limitations that were observed.

Keywords— Git, GitHub, Version Control, Feature Branch, Pull Request, Merge Conflict, Team Collaboration

I. INTRODUCTION

A. Background

One such technique that separates an organized engineering team from those who merely happen to code the same software is version control. Git is a distributed open source version control system developed in 2005 by Linus Torvalds, which logs all the changes made to a project and enables multiple people to make changes to the same software

without inadvertently overwriting one another's code. Whereas, GitHub is a different piece of software that works on web and offers more collaborative capabilities than Git itself. It has pull requests, discussion forums and bug reports in addition to Git repos. This case study analyzes how a team leveraged these two pieces of software to develop a socially useful application: Token and Queue Management System for the OPD of a Government Hospital.

B. Importance of Git and GitHub for Software Development

Software development for the healthcare industry can be viewed as an example of why version control is important as any error in development or modification of the software might lead not only to wasting of time but also might do harm to the patients affected in the queue. Git and GitHub tools provide every individual developer with an offline-capable workspace and an ability to see all changes made in it as well as all the team with one source of the truth. Thus, a small group of developers works faster without losing an ability to check, question and correct the change before putting it into the working version.

C. Objectives of the Case Study

- To explain how the small group of developers used Git during one real-life development cycle of the software product.
- To analyze the strategy of branches creating and pull request usage of the group.
- To investigate the merge conflict between two developers working on the same queue-ordering function

D. Scope of the Study

This research only covers the collaboration process and the source code of a single product team and does not cover the procurement process of hospital IT, data privacy of the patients, or the clients of the company. The names of the

CASE STUDY REPORT

branches, the source code, and the conflict sample presented in this paper have been specifically created for this case study and did not come from the repository of any actual company.

II. COMPANY BACKGROUND

A. Company Overview

Ashva Technologies Pvt. Ltd. is a fictional software development firm having fourteen employees which has been created for this case study and works from a cell of Technology Business Incubator (TBI) of an university located in Ahmedabad, Gujarat. Ashva Technologies specializes in developing customized digital governance solutions for municipal corporations and government hospitals, but not enterprise solutions.

B. QNext Project

QNext is the software application that was deployed in the entire case study. This web-based software application provides queue and token management services in the Outpatient Department (OPD) of a government hospital. Once a patient has made his/her booking for the token, he/she will be provided with an estimated waiting position on the queue and will be taken to the respective consultation room once it is his/her turn. The senior citizens, aged 60 and above and those who have marked themselves as emergencies are pushed forward in the queue without compromising on the waiting process of other patients. The pilot deployment being referred to in this case study is an imaginary name given to the hospital which is known as "Sanjivani General Hospital, OPD Block-B" and not any real hospital. The backend of the software application is written in Node.js and Express and React for frontend, and MongoDB is used for database.

C. Business Requirement for Collaborative Development

In the QNext project, the team includes four developers and one QA engineer that need to develop three modules including token booking, live queue dashboard, and SMS alert in only three weeks before the hospital pilot reviews these components. In order to do this, several people had to work parallelly on different but correlated parts of the same queuing logic without any dependency among each other.

D. Need for Utilizing a Git Workflow

Before using Git and GitHub, coordination among team members was informal, which led directly to the problems mentioned above in Section III. Therefore, Git workflow was necessary for developers to work in an isolated environment, a version history of changes made, and the review phase before integration into the final version of QNext.

III. PROBLEM ANALYSIS

A. Current Development Process (Pre-Git)

Before the use of Git, the QNext team would transfer their codes by employing one Google Drive folder and one WhatsApp group to talk with their mentor from the TBI division. Once the coding was complete, they would create a zip file from the entire project and upload the same, thus overwriting whatever existed before. There was no branch nor history of any commits – it was a mere merging of changes upon uploading a new zip file.

B. Problems Encountered

One problem occurred in relation to this very informal manner during the initial two weeks of the development process. Kavya had spent nearly three hours working on the first draft of the SMS alert module. As she was still engaged in doing so, Yash uploaded another version of the zip file of the whole project in order to fix an issue that he was facing with the dashboard without realizing that in his copy of the project there were no changes that Kavya had made.

- Inconsistent writing: if any two people saved to the common folder, it would overwrite each other's changes, similar to what happened with Kavya's SMS module.
- No change history: there was no way to track which person made changes to a certain line, when and why.
- Complex rollbacks: fixing a problem required going into past versions stored as zip files and trying to find a correct one based on name and date.
- No structured review process: code would get into the common folder without having anyone else check it before.
- Lack of visibility for the team lead: Aditya did not have a way to determine what was being done at the moment and how far the module was from completion.

C. Functional and Non-functional requirements

The project needed to have a tool which could ensure that the development team had the ability to work independently on each module, version control on every change made, the ability to do code review prior to integrating the code, roll back to any previous version and be good enough to work with a limited number of developers at no cost.

D. Constraints and Assumptions

The constraints and assumptions were that there was an internet connection available for push/pull of code; a free GitHub account was enough for a private repository of this size; and that the technology stack would not have to be changed to accommodate the new process.

CASE STUDY REPORT

TABLE I

Development Practice Before and After Adopting the Git-Based Workflow

Aspect	Before Git-Based Workflow	After Git-Based Workflow
Code storage	Zipped project folder on a shared Google Drive	Central Git repository hosted on GitHub
Change tracking	No history; changes were silent	Every change recorded as a commit with author and message
Parallel work	Kavya's SMS module was overwritten by Yash's upload	Isolated feature branches per developer/module
Review	None; zip uploads went live directly	Mandatory pull request review before merge
Rollback	Manually searching old zip files by date	git revert / checkout to any earlier commit
Conflict handling	Silent overwrite, discovered a day later	Conflicts detected explicitly and resolved before merge

IV. GIT & GITHUB SOLUTION ANALYSIS

A. Chosen Tools: Git and GitHub

Git and GitHub are the chosen tools for the team to solve its version control problem. These two are not the same. While Git is the actual version control tool working at the developer's computer with commits, branches and project history on the local level, GitHub is the additional online platform that provides hosting and collaboration functions such as pull requests, reviews, issues management, permissions and other functions based on Git. While working with Git the developer does not need an Internet connection at all; GitHub is required only if the developer intends to collaborate with others on his work.

B. Why Git and GitHub Were Chosen

- All developers have a complete repository of all previous changes locally, and thus development can happen anywhere without an Internet connection.
- Branching in Git is relatively light, and that is why it is possible for everyone, including every bug fix, to have its own branch.
- The pull request feature in GitHub enabled the team's TBI mentor to look quickly at the changes without reading the actual code.
- They are both widely used in the software industry and, therefore, the process that the team created is directly applicable to work in the real world.

C. Licensing

Git is available under the GNU General Public License v2 (GPLv2), which ensures that Git is free and open source. GitHub, on the other hand, is a commercial product owned by Microsoft Corporation. It is based on Git technology, but GitHub itself is not an open source product. This is important in our case since the version control tool we are using (Git) is open source whereas the hosting tool (GitHub) is a paid service we use for free.

D. Comparison with Alternative Approaches

TABLE II

Comparison of Approaches Considered by the Team

Approach	Nature	Key Limitation for QNext
Shared Drive folder / zip upload	No version control	No history, easy to overwrite, exactly what caused the SMS-module incident
Centralized VCS (e.g., Subversion)	Single central repository	Requires constant network access; branching is heavier to set up
Git + GitHub (adopted)	Distributed VCS with hosted collaboration	Requires the team to learn Git commands and pull-request discipline

E. Advantages of Using Git and GitHub

The advantage that the team immediately realized was obvious and clear. Two team members could now simultaneously work on the same file on the same day without one person's changes mysteriously vanishing. Finally, the reviewing stage provided a light way for the team's mentor, who does not write code every day, to be aware of weekly changes.

V. SYSTEM ARCHITECTURE AND BRANCHING STRATEGY

A. General Git Cooperation Procedure

See Figure 1 for the process all QNext features go through starting from assigning a module to a developer at the Monday meeting until releasing tested code from the main branch.

CASE STUDY REPORT

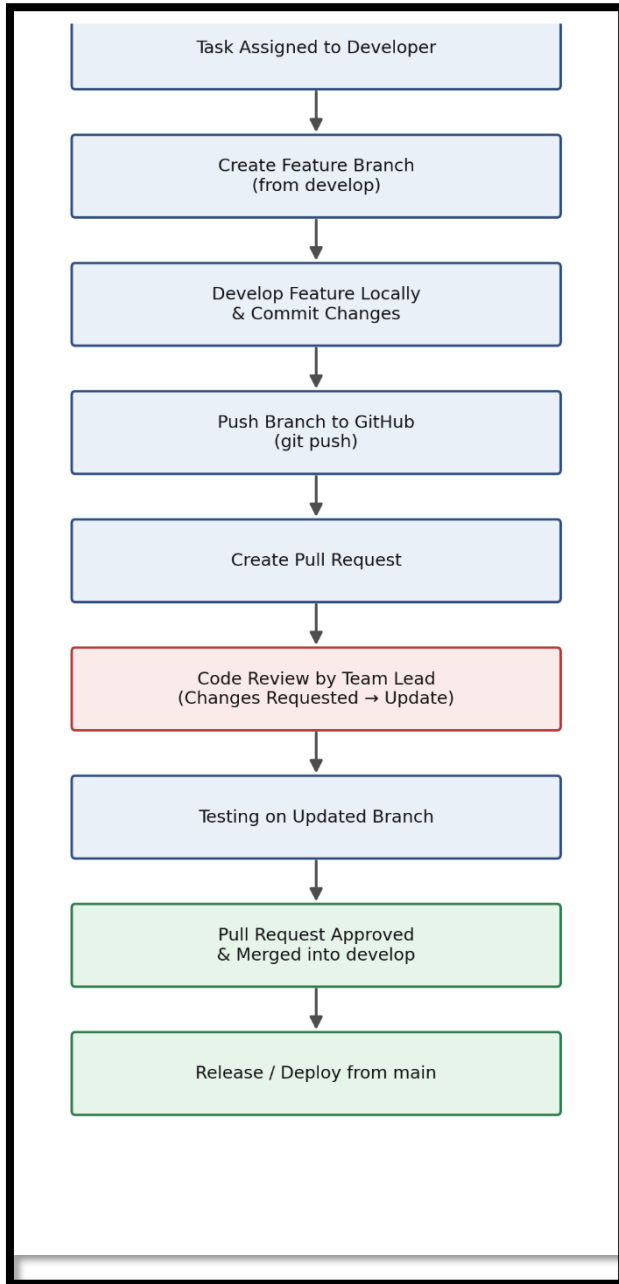


Fig 1: Overall Git collaboration workflow followed for every module in the QNext project.

B. Branching Strategy for QNext

There is a two-level branching strategy employed by the team which involves the use of a master branch and a development branch with temporary creation of branches based on a particular module. Refer to Figure 2 for the branching strategy used during the development of the above mentioned modules.

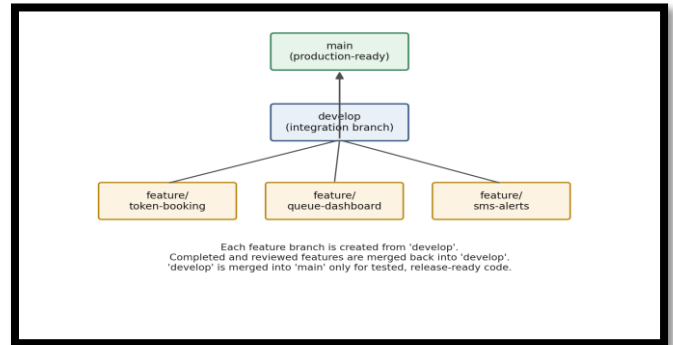


Fig 2: Branching strategy used for the QNext project.

TABLE III

Branches Used in the QNext Repository

Branch	Purpose	Example Work
main	Always reflects the version currently deployed to the hospital pilot server	Only receives merges from develop after a release is tested
develop	Integration branch where completed modules are combined and tested together	Receives merged feature branches before a release
feature/token-booking	Isolated branch for patient OTP login and token generation	Meera: OTP verification, calculateQueueOrder() logic
feature/queue-dashboard	Isolated branch for the live queue display shown to OPD staff	Yash: dashboard UI, priority reordering for senior citizens
feature/sms-alerts	Isolated branch for SMS notifications when a token is close to being called	Kavya: SMS gateway integration

C. Creation of Local and Remote Repositories

Every developer stores their local repository on their respective laptops, either through the first time use of git clone command against their GitHub repository, or using git pull / git fetch command on other days for fetching latest updates from the remote repository. A developer uses git commit command to commit their changes to their local repository and pushes the committed changes to GitHub using git push command, wherein git fetch command fetches new commits but does not merge them whereas git pull merges them in one single command. The difference between the local repository (private, can work offline) and remote repository (shared, always accessible to all) is the crux of Git as a distributed version control system.

CASE STUDY REPORT

VI. TEAM STRUCTURE AND REPOSITORY STRUCTURE

A. Team Structure

Table IV gives the names of the five members of the QNext team along with their responsibilities regarding Git/GitHub activities.

TABLE IV

Team Roles and Responsibilities

Role	Responsibility	Git/GitHub Activity
Aditya Rathod – Team Lead	Coordinates the sprint, reviews and approves pull requests, manages releases	Reviews PRs, merges develop into main, creates release tags
Meera Solanki – Backend Developer	Builds patient OTP login and token generation	Works on feature/token-booking; opens and updates PRs
Yash Trivedi – Frontend Developer	Builds the live queue dashboard for OPD staff	Works on feature/queue-dashboard; opens and updates PRs
Kavya Bhatt – Full-Stack Developer	Builds the SMS alert module	Works on feature/sms-alerts
Om Prajapati – QA Engineer	Tests feature branches before and after merge	Pulls feature branches locally, comments on pull requests

B. Repository File Organization

The QNext repository contains two top-level folders for the backend and frontend (/server and /client) respectively, the common /services folder containing code used by multiple modules like the tokenService.js module mentioned in the conflict highlighted in Section VIII, and a .gitignore file placed in the root of the repository to ensure environment files are not committed into it.

C. Branch Protection and Commit Message Writing

Both main and develop branches are protected in GitHub meaning that they cannot be updated using a push but instead require a pull request to be approved before updates are pushed in. Developers should always aim to write descriptive commit messages such as “Add senior-citizen priority check to calculateQueueOrder”.

VII. FEATURE DEVELOPMENT, PULL REQUEST AND CODE REVIEW PROCESS

A. Feature Development: Token Booking and Queue Dashboard

Let us examine sprint 4 of the QNext project (03–14 March 2026), where the developers were required to get the patient token booking and live queue dashboard functionality fully implemented prior to the hospital pilot review. Meera forked a branch called feature/token-booking off of develop and started implementing the patient login using OTP and the algorithm that allocates a token to each patient. Meanwhile, Yash forked another branch called feature/queue-dashboard off of develop and worked on developing the live queue dashboard screen for the staff. B. Creating and Pushing Feature Branches

```
git checkout develop
git pull origin develop
git checkout -b feature/token-booking
... edit files, implement OTP login & token logic ...
git add .
git commit -m "Add OTP verification and token generation"
git push origin feature/token-booking
```

C. Pull Request Creation

Since Meera had tested feature/token booking without finding any issues in it, she created a pull request on GitHub for merging it with develop. The description of the pull request included changes that were made in the code and reasons and GitHub showed those lines which were modified in/from develop.

D. Code Review Process

As a team lead, Aditya reviewed the pull request at the team's Tuesday sync meeting and used the screen sharing option to do it through the "Files changed" tab. He provided a code review comment asking Meera to provide the case for OTP request failure and then marked the pull request as “Changes requested”. This resulted in creating another pull request by Meera, which got automatically merged with the previous pull request and got approved by Aditya.

TABLE V

Typical Pull Request Stages for a QNext Feature

Stage	Activity	Responsible Person
1	Open pull request from feature branch to develop	Feature developer
2	Automatic diff and file-change summary generated	GitHub
3	Code review and inline comments	Team lead
4	Requested changes addressed and re-pushed	Feature developer
5	Approval given	Team lead
6	Branch merged into develop	Team lead

CASE STUDY REPORT

VIII. MERGE CONFLICTS SCENARIO AND RESOLUTION

A. Causes of the Merge Conflict

Both the Token booking service module and the Queue dashboard service module had to determine which patients qualify as being "Next in Line". Hence, both developers had to make changes to the same method, namely, `calculateQueueOrder()` in the shared file named `src/services/tokenService.js`. In her case, Meera implemented the method in such a way that it returned the list of patients based on the sequence in which tokens are generated, which is "First-In First-Out". Yash, however, had to modify this method while working on developing the dashboard service in such a way that it made patients who have flagged as Senior citizen or Emergency move ahead of other patients in the queue.

B. Sequence of Events

- Meera gets her pull request checked on Tuesday sync call and merges her code into develop after approval, thereby changing the file `tokenService.js` on develop.
- Then Yash fetches and merges develop code into his local `feature/queue-dashboard`, and his `feature` branch code has Meera's merged code of the function.
- Since Yash has edited the same lines of `tokenService.js` on `feature/queue-dashboard` branch and it is not yet committed, there is a conflict and merge fails.

C. File Involved and Conflicting Changes

The conflict appears inside `src/services/tokenService.js`, in the `calculateQueueOrder` function. Meera's version (already on develop) sorts strictly by token number. Yash's version (still only on his branch) sorts by a priority flag first. Git inserts conflict markers directly into the file at the point of disagreement:

```
function calculateQueueOrder(tokens) {
<<<<<<< HEAD
// Meera: simple first-in, first-out ordering
return tokens.sort((a, b) => a.tokenNumber - b.tokenNumber);
=====
// Yash: senior citizen / emergency patients first
return tokens.sort((a, b) => {
  if (a.isPriority !== b.isPriority) {
    return a.isPriority ? -1 : 1;
  }
  return a.tokenNumber - b.tokenNumber;
});
>>>>>>> feature/queue-dashboard
}
```

The part between "`<<<<<<< HEAD`" and "`=====`" indicates the current state in the develop branch (Meera's simple FIFO ordering), while the section between "`=====`" and "`>>>>>>>`" indicates Yash's conflicting

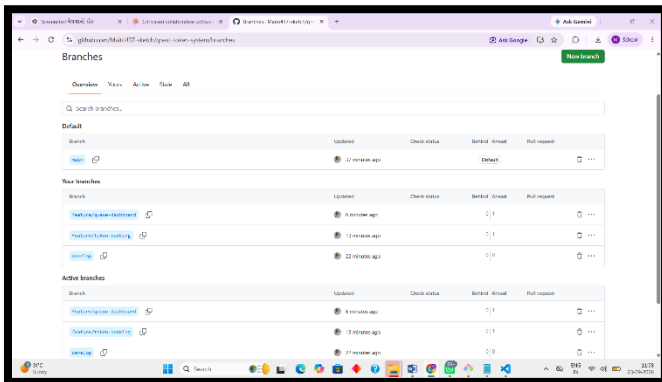


Fig 3: Branch list on GitHub showing all active QNext branches.

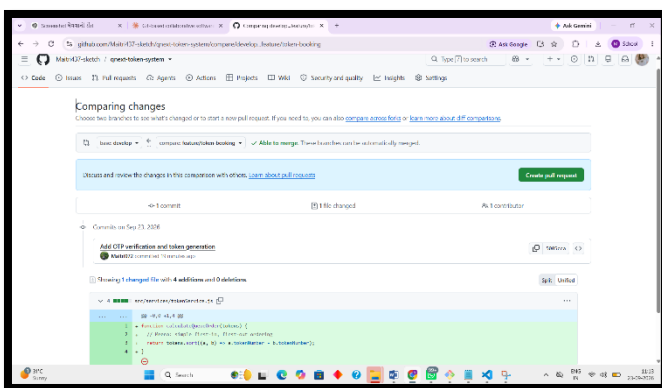


Fig 4: Pull request creation screen for `feature/token-booking` → `develop`.

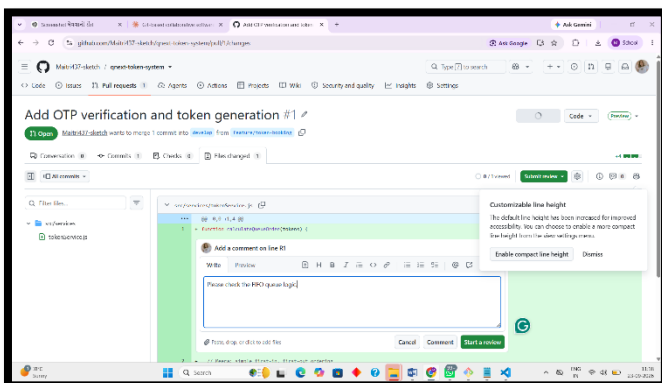


Fig 5: Code review comment on the `feature/token-booking` pull request.

CASE STUDY REPORT

version with priorities from his feature/queue-dashboard branch.

D. Problem Solution

1. Identifying the issue: GitHub identifies that “This branch has conflicts that must be resolved” and indicates tokenService.js as the conflicting file.

2. Upgrading local branch: Yash executes git fetch origin followed by git merge origin/develop on his local branch feature/queue-dashboard so that the conflict can be seen and solved locally, since it is much easier to solve.

3. Analyzing both versions: Yash analyzes the two parts, HEAD (develop) part and the other part which is his feature/queue-dashboard and realizes that none of the two are fully correct and that the queue dashboard should have both FIFO and prioritization.

4. Combining the solution: Yash retains his priority check but modifies the code such that the patients are first arranged according to their priority flag and then arranged by token number in each group of priority.

5. Solving conflicts through removal of conflict markers: The <<<<<<<, =====, and >>>>>>> lines are stripped out of the code and a single function emerges.

6. Solution testing: Om, the QA engineer, gets immediately involved; he creates manual tokens like the senior citizen token and makes sure that the merged logic will queue up the patients correctly.

7. Solution commit creation: The merged file is then staged and committed using git commit -m "Resolve merge conflict: merging FIFO and priority logic in calculateQueueOrder".

8. Pull request update: Yash commits his solution to the feature/queue-dashboard branch and updates the pull request.

9. Pull request review and approval: Aditya reviews the logic, confirms that it satisfies both the booking and dashboard requirements, and approves the pull request.

10. Successful merge: feature/queue-dashboard gets merged to develop and the merged queue logic gets committed to the codebase.

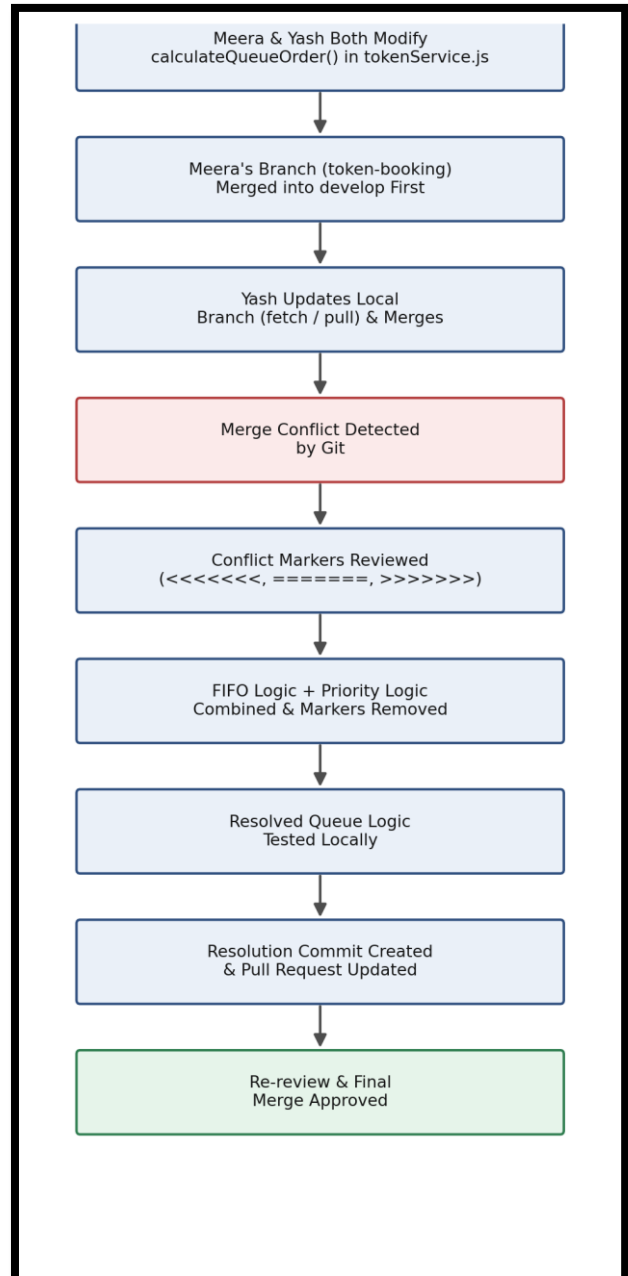


Fig 6: Merge conflict detection and resolution flow for calculateQueueOrder() in tokenService.js.

CASE STUDY REPORT

TABLE VI

Summary of the Merge Conflict Encountered in the QNext Project

Situation	Cause	Impact	Resolution
Conflict in src/services/tokenService.js between feature/token-booking and feature/queue-dashboard	Both branches edited the same lines of the shared calculateQueueOrder function after branching from the same point in develop	Yash's pull request could not be merged automatically; the dashboard module was blocked until resolved	FIFO and priority-based ordering combined into one function, conflict markers removed, logic re-tested by QA and re-reviewed before merging

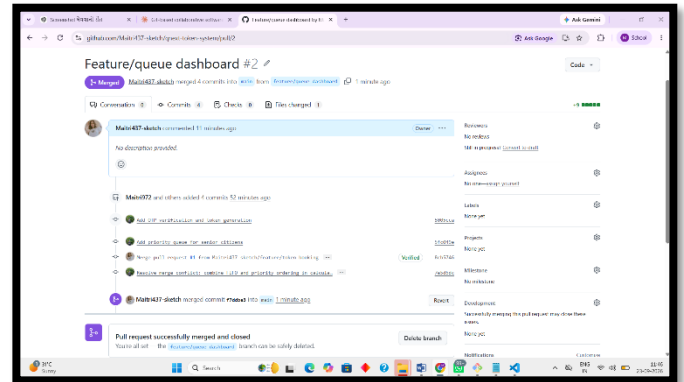


Fig 9: Successful merge confirmation on GitHub.

IX. TESTING AND VERSION CONTROL PRACTICES

A. Testing the Resolved Code

Prior to merging a pull request, Om pulls the feature branch to his computer and manually navigates the screens impacted by the bug along with running the test cases already implemented in the project. As far as resolving the issue mentioned in Section VIII, Om implemented a set of test tokens for both regular users and senior citizens and verified that the newly merged calculateQueueOrder function properly ordered them.

B. Tracking Versions and Performing Revert

As all changes in the codebase are stored in commits, the team can utilize git log command in order to see the full history of changes made and reasons for doing so. Should a change introduced by a merge turn out to be erroneous (for example, the priority algorithm is found to favor regular patients), it can be undone using git revert command, which creates a new commit reverting the change, but not erasing the history of the project. In contrast to the version control system used before Git, such operation would involve manually searching the older zip files.

C. Tags and Release

After testing the code that has been developed and it has been determined that it is ready for the hospital pilot program, Aditya merges the code into main and tags the commit with a Git tag such as v0.4.0-pilot. GitHub converts this tag into a "Release," and gives the team a way to pinpoint an exact location in their history if there are any issues with that release.

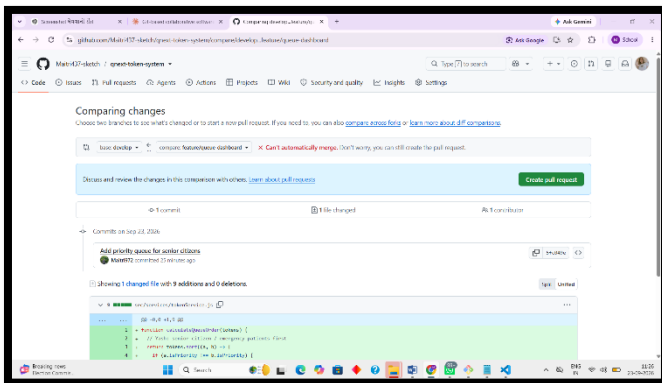


Fig 7: Merge conflict warning shown on the feature/queue-dashboard pull request.

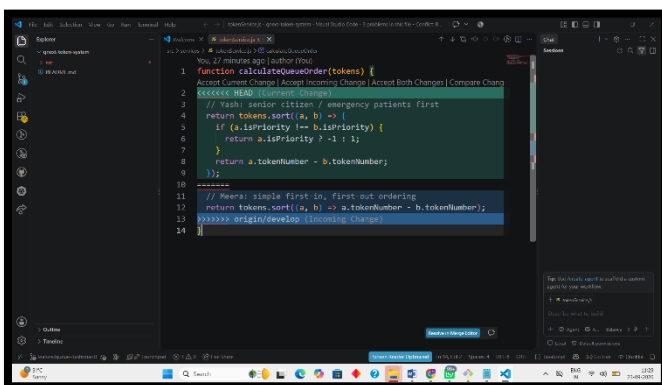


Fig 8: Local conflict-resolution view in the code editor.

CASE STUDY REPORT

X. CO2 AND SDG MAPPING

A. CO2 – Version Control Tools and Collaboration

Course Objective CO2 requires students to display knowledge on the use of version control systems. The Table VII below links the various uses of Git/GitHub in this case study to Course Objective CO2.

TABLE VII

Mapping of Case Study Activities to CO2

CO2 Sub-Area	Demonstrated Through
Git fundamentals	Local/remote repository setup, commit, push, pull, fetch, clone (Section V-C)
Branching	main / develop / feature branch structure (Section V-B, Table III)
Pull requests & code review	Token-booking and queue-dashboard PR workflow (Section VII)
Merge conflicts	calculateQueueOrder conflict, detection and resolution (Section VIII)
Version history & rollback	git log, git revert, tags and releases (Section IX)
Team collaboration	Defined roles across five team members (Table IV)

B. SDG 9 - Industry, Innovation and Infrastructure

SDG 9 aims at constructing robust infrastructure and innovation. On the level of this project, the Git flow ensures robustness of the software infrastructure underlying the public health tool because any modifications made to the queuing system can be tracked, reviewed and reverted, and the fact is crucial for this project because this system determines the order of visits to the clinic for patients. Branching and pull requests allow for faster and safer iterations as recommended by SDG 9 in the sphere of technology, but the object is a digital public health tool rather than a commercial product.

C. SDG 17 - Partnerships for the Goals

SDG 17 promotes partnerships and exchange of resources for achieving a common goal. QNext is a partnership of a small technology firm and a public hospital with the aim to improve public services; on the level of engineering process, the common repository on GitHub, pull request reviews and branching are the tools allowing for collaborative work of four developers and their mentor towards one result.

XI. RESULTS, ADVANTAGES AND CHALLENGES

A. Identified Results

- All changes made to the QNext repository can be traced back to a particular commit, author, and pull request.
- Silent overwrites like the one that wiped out Kavya's work in the SMS module cannot happen since every programmer works on his own branch.
- The calculateQueueOrder conflict that was covered in this case study was instantly detected and solved using Git.
- Aditya is fully aware of what is being developed, what is being reviewed, and what is merged based on the status of open branches and pull requests.
- There are more errors being caught in code reviews rather than during integration.

B. Advantages

The most obvious advantage was that it allowed two developers to make changes to similar code in the same file on the same day without one of their changes being erased without notice. The review process also took on some characteristics of information sharing; Meera, for example, came to understand the logic behind priority queues simply by having to review Yash's solution to the conflict.

C. Problems

- There was a definite learning curve for people on the team who had never worked with Git commands and branches before.
- Fixing the conflict in tokenService.js involved knowing both the FIFO requirement and the priority requirement, not just one of them.
- Pull requests that are left open for more than a day or two start to drift away from the develop branch, which makes future conflicts more likely.
- The whole process is useful only if commits and pull requests are specific; some early commits with vague descriptions such as "fix stuff" were hard to understand

CASE STUDY REPORT

XII. KEY LEARNING AND CONCLUSION

A. Case Study Summary

The QNext team was observed at Ashva Technologies Pvt. Ltd. during their transition from a unorganized approach using zip files to a well-organized approach that made use of feature branches, pull requests, code reviews, and a merge conflict resolution for the ordering logic of the hospital queue.

B. Technical Insights

The above case has helped to establish that both Git and GitHub have separate but complementary roles to play; Git as the distributed version control system existing in individual machines, while GitHub as the common platform that makes all the individual Git repository systems work in concert as a team. The above case further illustrates the fact that the occurrence of merge conflicts, such as that in calculateQueueOrder, does not mean that there is an error because what Git does is to refuse to make an assumption of the differences.

C. Learning Outcomes

By completing this case study, concepts like repository, commit, branch, push, pull, fetch, merge and pull request, which were theoretical definitions before, became practical steps for the team as they developed a tool that was going to be used by real patients.

D. Industrial Relevance

The use of a branching strategy, a pull-request review system and a conflict resolution process illustrated above is common practice in the software industry. This makes the case study relevant to the work environment a Computer Engineering graduate is going to find him/herself in after graduating, regardless of whether s/he is developing software for the government sector or elsewhere.

REFERENCES

- [1] Git, "Git Documentation," git-scm.com. [Online]. Available: <https://git-scm.com/doc>
- [2] S. Chacon and B. Straub, Pro Git, 2nd ed. Berkeley, CA: Apress, 2014. [Online]. Available: <https://git-scm.com/book/en/v2>
- [3] GitHub, "About pull requests," GitHub Docs. [Online]. Available: <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-pull-requests>
- [4] GitHub, "About merge conflicts," GitHub Docs. [Online]. Available: <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/addressing-merge-conflicts/about-merge-conflicts>
- [5] GitHub, "GitHub flow," GitHub Docs. [Online]. Available: <https://docs.github.com/en/get-started/using-github/github-flow>
- [6] United Nations, "Goal 9: Industry, Innovation and Infrastructure," Sustainable Development Goals. [Online]. Available: <https://sdgs.un.org/goals/goal9>
- [7] United Nations, "Goal 17: Partnerships for the Goals," Sustainable Development Goals. [Online]. Available: <https://sdgs.un.org/goals/goal17>